Do you need your own OAuth app?

A lookup tool that tells a Composio developer, in about thirty seconds, whether they need to register their own OAuth app, how long approval really takes, and what they can start today.

Live	composio-oauth-guides.vercel.app
Repo	github.com/chayan171999/Composio-project
Stack	Next.js (App Router) · TypeScript · Tailwind · Zod · Gemini · Upstash Redis · Vercel

The problem

Composio offers managed OAuth for many toolkits, but a developer needs their own OAuth app once they want their own branding on the consent screen, scopes beyond the defaults, a dedicated rate-limit quota, or faster polling triggers. Finding out what that costs is opaque: sometimes ten minutes, sometimes six weeks and a paid security audit. You usually discover which only after you are committed.

This answers five questions up front: do you need your own app at all; how long until production and what kind of blocker stands in the way; what you can start today so approval waits run in parallel with building; which steps an agent can do versus a human; and how confident each individual claim is.

What is built

Area	State
Seed guides	Three hand-verified: GitHub (no review), Google/Gmail (scope review), Meta/Facebook (business verification) — three different blocker types.
Live generation	Enter any app, the agent researches official docs and writes a guide in the same schema, streaming progress. Verified on Slack, Zoom, Figma and Linear.
Evidence	Every factual statement is a claim with a status (tested / sourced / inferred / unverified), its sources, and the date checked.
Persistence	Generated guides are written to Upstash Redis, so /guides/<slug> URLs are shareable and a repeat lookup costs nothing.
Search	Matches name, slug, aliases and website domain, so a pasted URL resolves. An existing guide is shown rather than regenerated.

Key decisions

The blocker type and the critical path lead the guide

Most integration docs are ordered by the provider's process, which is the wrong order for someone deciding whether to start. What a developer needs first is the shape of the obstacle, because that decides whether this is an afternoon or a quarter. The critical path follows, capped at four items chosen so approval waits run in parallel with building.

Composio's managed status is the first question, not a footnote

The most valuable answer this can give is often that you do not need to do any of this. The verdict leads with that, so the guide can talk you out of the work.

Evidence is per-claim, not per-page

A guide that is 90% accurate is dangerous if you cannot tell which 90%. Claim references render as footnote markers; hovering one shows the statement, its sources and when it was checked. An estimate never renders as confirmed fact.

Only three seed guides, deliberately

The slow part is not writing JSON. It is reading primary sources and being honest about what they do and do not say. Three genuinely verified guides demonstrate the evidence model better than twenty plausible ones.

How the agent works

Three bounded model calls, streamed to the browser over SSE as they run.

- **Discovery** — propose up to six likely official documentation URLs. Mechanical, so it runs on the cheaper Flash-Lite model.
- **Read** — fetch those pages with Gemini's URL-context tool and write a research brief with a source line per fact.
- **Composio check** — done in code rather than by the model: the Composio API when a key is configured, otherwise the public toolkit docs page.
- **Synthesise** — produce the guide schema as structured JSON, validated with Zod, with one repair attempt on failure.

What stops it inventing things

Prompting is not a guarantee, so the constraints are enforced in code:

- **Citations are an allow-list** built from Gemini's record of which pages were actually retrieved. A URL that 404s cannot be cited, so the model cannot invent a plausible docs link.
- **Generated guides may never claim *tested***. That status is reserved for things a human actually did, and validation rejects it outright.
- **Dangling claim references are rejected** — every reference must resolve to a real claim.
- **A sourced claim must cite a source**, and that source must be on the allow-list.

Anything failing these goes back for one repair pass. If it still fails, the generation errors rather than shipping a guide that quietly breaks its own rules. In the Zoom run, four of eight proposed URLs failed to retrieve; the missing topics surfaced as open questions instead of invented facts.

Verification

Full log in `docs/verification.md`. Researching the seed guides turned up two errors in the source material:

Claim	Initial research	Primary source
CASA assessment fee	~$540–$3,000, third party	**$500–$4,500** — Google's own docs
CASA duration	"~6 weeks"	**2–3 weeks tier-2, 4–6 weeks tier-3**
Nango source URL	cited	**404** — relocated to current address

Deliberately left unverified. Meta publishes no review SLA. Rather than quote an anecdotal community figure, that claim is marked *unverified* and the verdict says "Unknown" outright.

Agent output spot-checked. The generated Zoom guide's claims were checked by fetching the cited page: access tokens expire after one hour, refresh tokens after 90 days with rotation, PKCE required for loopback URIs, deauthorization endpoint required. All confirmed; no hallucination found.

Deviations from the brief

Gemini instead of the Anthropic API. The brief specified Anthropic with its server-side web search tool, and it was built that way first. It was switched because the Anthropic API has no free tier, and avoiding paid accounts was a harder constraint.

URL retrieval instead of web search. Google Search grounding returns a hard 429 on keys without billing — it has no free quota at all. So the agent discovers candidate documentation URLs and fetches them directly. This turned out to suit the problem better: Gemini reports exactly which URLs it retrieved, which is what makes the citation allow-list enforceable. Searching would have given the model a pile of results and no way to prove which page a claim came from.

Limits

- **Free-tier quota is the binding constraint.** Gemini allows roughly 20 requests/day on Flash, which is about 5–7 live generations. Existing guides cost nothing to view, which is most of why persistence matters. On exhaustion, generation fails with a message naming the reset window and the rest of the site keeps working.
- **Free-tier Gemini is flaky under load.** Transient 503s are common on the heavier document-reading call; calls retry with backoff, but a reviewer may need to try twice.
- **The rate limiter is per-process.** On Vercel each serverless instance keeps its own counter, so the real limit is looser than it looks.
- **There is no self-review pass.** The brief listed one as optional; it is not built. Hallucination is mitigated structurally rather than by asking a second model whether the first was honest.
- **The model split is measured on one side only.** Running everything on Flash-Lite was compared against Flash and was clearly worse (5 sourced claims against 9). The current split has not been compared end to end against the all-Flash baseline — quota ran out mid-run.
- **Generated guides are not hand-checked** beyond the Zoom spot-check, though they are labelled as machine-extracted with their date.

What I would do next

- **Scale without lowering the bar.** Batch-generate across toolkits with a human review queue before publication. The evidence model already makes that reviewable — a reviewer reads the claims, not the prose.
- **Freshness checks.** Every claim carries a URL and a check date, so a scheduled job could re-fetch sources, diff them and flag guides whose evidence moved. One source URL already died during this build.
- **Use the Composio API for real scope data**, so "do you need your own app?" compares the provider's scopes against Composio's actual defaults rather than reasoning about them.
- **A browser agent for the setup attempt**, which would turn inferred time estimates into tested ones — the one evidence status nothing currently earns.

Setup: `npm install`, copy `.env.example` to `.env.local` and set `GEMINI_API_KEY` (free, no billing account), then `npm run dev`. Everything else is optional.